Post lab

Producer-Consumer Problem using Semaphores

1. AIM

To implement and demonstrate the **Producer-Consumer problem** (also known as the BoundedBuffer problem) using **semaphores** in C, illustrating inter-process synchronisation and mutual exclusion in an Operating System environment.

2. OBJECTIVES

1. Understand the concept of process synchronisation and the critical section problem.
2. Study the role of semaphores (binary and counting) as a synchronisation primitive.
3. Implement the Producer-Consumer solution using POSIX semaphores in C.
4. Analyse how race conditions and deadlocks are prevented using proper semaphore usage.
5. Observe the behaviour of shared buffer access between producer and consumer threads.

3. THEORY

3.1 Introduction

The **Producer-Consumer problem** is a classic synchronisation problem in Operating Systems. It involves two types of processes sharing a fixed-size buffer: the **Producer**, which generates data and places it into the buffer, and the **Consumer**, which removes data from the buffer for processing. Without proper synchronisation, race conditions can occur.

3.2 Semaphores

A **semaphore** is an integer variable used for process synchronisation, accessed only through two atomic operations — **wait()** (P) and **signal()** (V). Two types are used here:

• **Counting Semaphore (empty, full)** — tracks the number of empty and filled slots in the buffer.

• **Binary Semaphore / Mutex** — ensures mutual exclusion while accessing the shared buffer.

3.3 Semaphore Definitions

Semaphore	Initial Value	Purpose
empty	N (buffer size)	Counts empty slots in buffer
full	0	Counts filled slots in buffer
mutex	1	Mutual exclusion on buffer access

3.4 Algorithm

Producer Process:

```
while (true) {     produce_item();          // Produce
an item     wait(empty);              // Decrement
empty count     wait(mutex);              // Enter
critical section     add_to_buffer();          // Place
item in buffer     signal(mutex);            // Exit
critical section     signal(full);             //
Increment full count
```

```
}
```

Consumer Process: `while (true) {`

```
wait(full);              //
Decrement full count     wait(mutex);             // Enter
critical section
    remove_from_buffer();    // Take item from buffer
signal(mutex);           // Exit critical section
signal(empty);           // Increment empty count
consume_item();          // Consume the item
}
```

■ 4. PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full, mutex;

void *producer(void *arg) {     int item;     for (int i = 0; i < 10; i++)
{         item = rand() % 100;         // Produce item
sem_wait(∅);
// Wait if buffer full         sem_wait(&mutex;);            // Enter
critical section         buffer[in] = item;         printf("Producer:
Produced item %d at index %d\n", item, in);         in = (in + 1) %
BUFFER_SIZE;         sem_post(&mutex;);            // Exit critical
section         sem_post(&full;);             // Signal item added
sleep(1);
    }     return
NULL;
}
```

Implementation in C using POSIX Threads and Semaphores:

```
void *consumer(void *arg) {
    int item;     for (int i = 0; i < 10; i++) {
sem_wait(&full;);             // Wait if buffer empty
        sem_wait(&mutex;);            // Enter critical section
item = buffer[out];         printf("Consumer: Consumed item %d from
index %d\n", item, out);         out = (out + 1) % BUFFER_SIZE;
sem_post(&mutex;);            // Exit critical section
sem_post(∅);            // Signal slot freed        sleep(2);
    }     return
NULL;
}
```

```
int main() {
```

```
    pthread_t        prod,        cons;
sem_init(∅,      0,      BUFFER_SIZE);
sem_init(&full;,            0,      0);
sem_init(&mutex;,         0,        1);
pthread_create(∏,    NULL,    producer,
NULL);     pthread_create(&cons;, NULL,
consumer, NULL);     pthread_join(prod,
NULL);       pthread_join(cons,  NULL);
sem_destroy(∅);
sem_destroy(&full;);
sem_destroy(&mutex;);     return 0;
}
```

■ 5. RESULT

Sample output observed after compiling and running the program:

```
Producer: Produced item 83 at index 0
Producer: Produced item 46 at index 1
Consumer: Consumed item 83 from index 0 Producer:
Produced item 17 at index 2
Consumer: Consumed item 46 from index 1 Producer:
Produced item 63 at index 3
Consumer: Consumed item 17 from index 2
Producer: Produced item 25 at index 4
Consumer: Consumed item 63 from index 3
```

```
Producer: Produced item 71 at index 0
Consumer: Consumed item 25 from index 4
...
```

Observation:
The producer and consumer operate concurrently without interfering with each other. The buffer never overflows (producer waits when full) and never underflows (consumer waits when empty).
Mutual exclusion is correctly maintained — no two processes access the buffer simultaneously.

■ 6. CONCLUSION

The Producer-Consumer problem was successfully implemented using **POSIX semaphores** in C. The experiment demonstrated:

- ✓ Semaphores effectively prevent race conditions on the shared buffer.
- ✓ The *empty* and *full* semaphores manage buffer occupancy, avoiding overflow and underflow.
- ✓ The *mutex* semaphore ensures mutual exclusion during critical section access.
- ✓ The solution is deadlock-free when semaphores are acquired in the correct order.

This experiment strengthened the understanding of process synchronisation, a fundamental concept in Operating Systems essential for designing safe and efficient concurrent programs.